

Révision POO JAVA

Problème :

➔ Un compte est caractérisé par un login et un password, terminez le code manquant :

```
public class Compte {
    private String login;
    private String password;
    public Compte(String login, String password){
        this.login = ..... ;
        this.password = ..... ;
    }
    // Constructeur de copie
    public Compte(Compte compte){
        this.login = ..... ;
        this.password = ..... ;
    }
    // on suppose que les getters & setters sont fournis en cas de besoin
    @Override
    public String toString() {
        return "login=" + ..... + ", password=" + ..... ;
    }
}
```

Un utilisateur est caractérisé par un code et un nom et **il possède un compte**.

Remarque :

Un compte ne peut pas exister sans l'utilisateur qui le possède. Cela veut dire que le compte existe uniquement tant que l'utilisateur existe. Si ce dernier est supprimé, son compte est automatiquement supprimé.

En POO, on représente cette relation à l'aide d'une composition. La classe **User** (le composé) est responsable de créer et gérer le **compte** (le composant).

➔ Soit la classe abstraite **User** dont l'un des attributs est de type **Compte**. Complétez avec le code manquant.

```
public ..... class User
```

```
{
    private int code;
    private String nom;
    private Compte compte;

    // Dans une relation de composition, l'objet User doit créer lui-même son compte

    public User(int code, String nom, String login, String password) {
        this.code = code;
        this.nom = nom;
        this.compte = new .....;
    }
    // Lors de la copie, l'objet User (copie) doit créer son compte en faisant une
    copie(profonde) // du compte et non partager la même référence.
    public User(User user)
    {
        this.code = user.code; this.nom = user.nom;
        this.compte = new .....;
    }
    // les getters & setters sont fournis
    @Override
    public String toString() {
        return "code=" + code + ", nom=" + nom + ",
            compte= {" + .....+"}";
    }
}
```

➔ Soit l'interface **IMembre** qui définit les capacités des membres. Complétez avec le code manquant.

```
public ..... IMembre
{
    public void ajouterPointFidélités(int nombre);

    public void retirerPointFidélités(int nombre) throws PointsFidélitésException;

    static public int convertirPFenDH();

    public void afficher();
}
```

➔ Soit l'exception personnalisée **PointsFidélitésException**. Complétez par le code manquant :

```
public class PointsFidélitésException .....
{
```

```
public PointsFidélitésException(String message) {
    ..... ;
}
}
```

- ➔ Soit la classe Membre qui hérite de la classe **User**, et qui implémente l'interface **IMembre**.
Complétez avec le code manquant.

```
public class Membre .....
{
    private int pointFidélités;
    private static int totalPointsFidelités = 0; //total de tous les membres
    public Membre( int code, String nom, String login, String password, int pointFidélités)
    {
        ..... ;
        this.pointFidélités = pointFidélités;
        totalPointsFidelités = .....;
    }
    @Override
    public void ajouterPointFidélités(int nombre){
        ..... ;
        ..... ;
        ..... ;
    }
    @Override
    public void retirerPointFidélités(int nombre) throws PointsFidélitésException{

        if (nombre>this.pointFidélités)
            ..... ;
            ..... ;
            ..... ;
        }
        // 15 points est équivalent à 1 DH
        @Override
        public int convertirPFenDH(){
            return pointFidélités/15;
        }
    }
}
```

```

@Override
public String toString() {
    return .....;
}
@Override
public void afficher()
{
    System.out.println(.....);
}
}

```

➔ Complétez le main par les opérations demandées :

```

public static void main(String[] args) {
    // Création d'un membre
    IMembre estMembre = new Membre(12, "Alaoui", "A12000", "success4ever", 10);
    estMembre.ajouterPointFidélités(20);
    System.out.println("valeur en dirhams des points de fidélités :"+
    estMembre.convertirPFenDH());
    estMembre.afficher();
    // Appeler la méthode retirerPointFidélités(100) sur estMembre puis afficher

```

```

.....
.....
.....
.....
.....
.....
.....
.....
.....
}

```